

Toward Hardening VirtIO Drivers for Confidential VMs with a Rust-based DSL

Lianjie Wang
The University of Tokyo
Tokyo, Japan
lian@is.s.u-tokyo.ac.jp

Takahiro Shinagawa
The University of Tokyo
Tokyo, Japan
shina@is.s.u-tokyo.ac.jp

EXTENDED ABSTRACT

Confidential Virtual Machines (CVMs) such as AMD SEV-SNP and Intel TDX exclude the host hypervisor from the Trusted Computing Base (TCB), so that sensitive workloads remain protected even from a malicious cloud provider. However, the guest must still rely on the untrusted host for I/O, typically through VirtIO [9]. This inverts the conventional trust relationship: the VirtIO device is now adversarial, whereas most existing drivers predate such a threat model and implicitly trust the data the device places in shared memory. For example, device-supplied length fields are often used without bounds checks, and validated fields may be re-read from shared memory, which the host can mutate at any time (double fetch). A malicious host can thus feed malformed inputs to trigger buffer overflows or Time-of-Check-to-Time-of-Use (TOCTOU) attacks, subverting the guarantees the CVM provides. Such vulnerabilities are not hypothetical: recent CVEs affect the Linux drivers for virtio-vsock [1], virtio-net [2], and virtio-bt [3].

Prior work hardens the drivers themselves. Distrust is retrofitted into mainline Linux check by check, guided by audit and fuzzing [6], but such retrofitting is error-prone: 12 of over 40 VirtIO hardening commits revert or amend earlier ones [8]. Gramine-TDX [7] avoids this legacy with its own VirtIO drivers, which cache control fields against TOCTOU, but again by hand and per driver. Another line of work generates validated parsers for untrusted formats: Nail [4] and EverParse3D [11] derive memory-safe, even formally verified, C parsers, from separate specification languages, but Nail assumes an input buffer that the adversary cannot mutate, and EverParse3D relies on F* toolchains. AVID [5] is closest to ours and derives validators from struct definitions with Rust procedural macros, but its PEG grammars cannot express VirtIO’s length-dependent layouts. Rust for Linux [10] enhances memory safety in the Linux kernel, but provides no VirtIO bindings yet, and static typing alone cannot enforce protocol constraints or prevent TOCTOU.

To harden the VirtIO drivers by construction, we propose a framework for writing them in Rust on top of generic Rust for Linux bindings, which abstracts the VirtIO primitives into typed interfaces that driver code cannot misuse and that validate host input automatically. A Queue wraps the underlying virtqueue and owns the buffers lent to the host, returning ownership to the driver only after the device has released it. Since the host may keep writing to shared memory, the buffer is first exposed as a `VolatileSlice`, whose contents are reachable only through a forward-only cursor that consumes each byte at most once. On top of the cursor, a Domain-Specific Language (DSL) lets the driver author describe the packet layout as a plain Rust struct and declare per-field constraints such as length bounds, ranges, and enumerations. A procedural macro then derives from that struct a parser that fetches each field

exactly once into private memory and rejects the packet unless every constraint holds. Double fetches therefore cannot occur, unvalidated data never reaches the driver, and the same guarantees carry over to every driver built on the framework.

Using this framework, we have ported the Linux guest drivers for virtio-rng and virtio-vsock to Rust, and evaluated the robustness of the latter by fuzzing at two levels. In userspace, libFuzzer drives the DSL-generated parsers directly with random inputs, while an oracle independently re-checks every declared constraint. For the full driver path, a modified QEMU injects mutated VSOCK packets into the RX virtqueue for 24 hours; our driver rejected the malformed ones, processed the rest, and triggered no crash or KASAN report. The same QEMU also mutates the shared buffer after delivery to race the header fetches, which the forward-only cursor renders ineffective by construction. Finally, replaying CVE-2025-39718 panics the unpatched C driver, whereas our parser rejects the packet at the length field.

We measured host-to-guest VSOCK throughput over vhost-vsock against the original C driver on Linux 6.18 with an Intel i9-13900K, using 64 KiB, 4 KiB, and 256 B messages. With buffer size, virtqueue depth, and refill strategy all aligned with the C driver, the hardened Rust driver incurs a 2% to 7% overhead, reaching 33.3 Gbps at 64 KiB. With a refill strategy optimized to reduce host stalls, it instead outperforms C by 9% to 34%, reaching 40.3 Gbps. This refactoring is orthogonal to the hardening, but the ownership discipline of Queue has made it safer to attempt.

We are now porting virtio-blk, exercising the same abstractions on a different device class. The DSL is also usable without a rewrite: its generated validators are already called from the existing C driver through FFI, an incremental path for drivers not yet ported. We next plan coverage-guided kernel fuzzing for broader driver paths, and Miri to check the unsafe core of the framework itself.

References

- [1] CVE-2025-39718. vsock/virtio: Validate length in packet header before `skb_put()`.
- [2] CVE-2025-40292. virtio-net: fix received length check in big packets.
- [3] CVE-2026-46186. Bluetooth: virtio_bt: validate rx pkt_type header length.
- [4] Julian Bangert and Nickolai Zeldovich. 2014. Nail: A Practical Tool for Parsing and Generating Data Formats. In *Proc. OSDI ’14*.
- [5] Madeleine Dease Dawson. 2019. *Secure Input Validation in Rust with Parsing-Expression Grammars*. Master’s thesis. Massachusetts Institute of Technology.
- [6] Intel Corporation. 2026. Intel Trust Domain Extension Guest Linux Kernel Hardening Strategy. Accessed: 2026-08-02. <https://intel.github.io/ccc-linux-guest-hardening-docs/tdx-guest-hardening.html>
- [7] Dmitrii Kuvaiskii et al. 2024. Gramine-TDX: A Lightweight OS Kernel for Confidential VMs. In *Proc. ACM CCS ’24*. doi:10.1145/3658644.3690323
- [8] Hugo Lefeuvre et al. 2023. Towards (Really) Safe and Fast Confidential I/O. In *Proc. HotOS ’23*. doi:10.1145/3593856.3595913
- [9] OASIS Open. 2023. Virtual I/O Device (VIRTIO) Version 1.3.
- [10] Rust for Linux contributors. 2025. Rust for Linux. <https://rust-for-linux.com/>.
- [11] Nikhil Swamy et al. 2022. Hardening attack surfaces with formally proven binary format parsers. In *Proc. PLDI ’22*. doi:10.1145/3519939.3523708